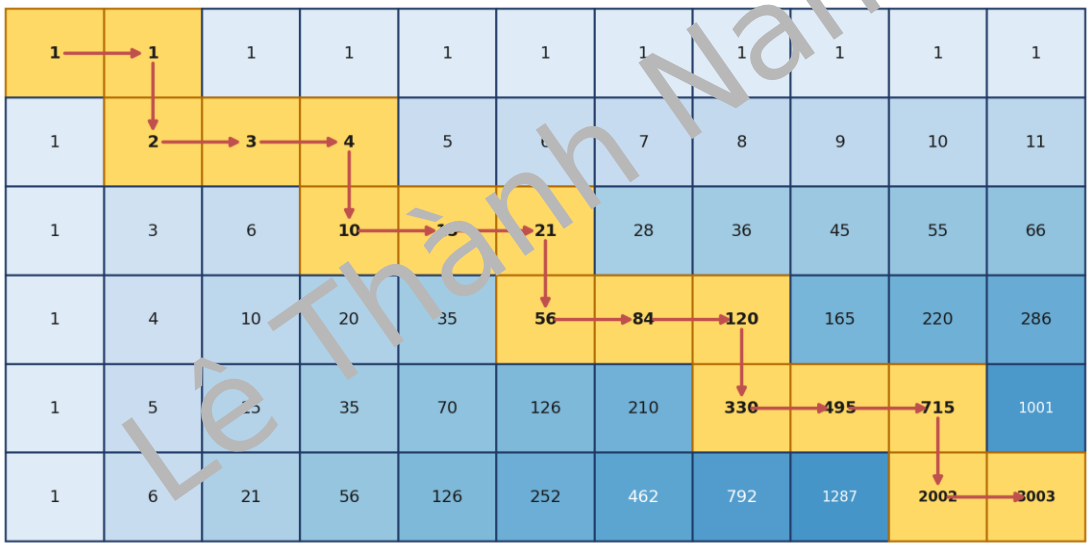
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

QUY HOẠCH ĐỘNG

Từ cơ bản đến nâng cao

TẬP 1 · LÝ THUYẾT



$$f(i, j) = f(i - 1, j) + f(i, j - 1)$$

Chương 1. NỀN TẢNG QUY HOẠCH ĐỘNG
Chương 2. CÁC MÔ HÌNH KINH ĐIỂN
Chương 3. QUY HOẠCH ĐỘNG NÂNG CAO
Chương 4. TỐI ƯU HÓA QUY HOẠCH ĐỘNG

Lê Thành Nam

LỜI NÓI ĐẦU

Trong đề thi học sinh giỏi Tin học, quy hoạch động gần như luôn có mặt, thường ở những câu phân loại nhất. Đây cũng là chủ đề mà nhiều học sinh thấy "khó vào" nhất: Đọc lời giải thì hiểu, nhưng tự gặp bài mới thì không biết bắt đầu từ đâu. Nguyên nhân là quy hoạch động không phải một thuật toán cụ thể để học thuộc, mà là một cách suy nghĩ: Chia bài toán lớn thành những bài toán con cùng dạng, giải mỗi bài toán con một lần, và ghép lại.

Bộ tài liệu này được viết để giúp em hình thành cách suy nghĩ ấy một cách có hệ thống, từ những bài đếm đơn giản nhất đến các kĩ thuật tối ưu hóa ở trình độ tiệm cận học sinh giỏi quốc gia. Bộ sách gồm ba tập đi cùng nhau:

- **Tập 1: Lý thuyết** (cuốn sách này) gồm 14 bài học chia thành 4 chương: Nền tảng quy hoạch động; Các mô hình kinh điển; Quy hoạch động nâng cao; Tối ưu hóa quy hoạch động.
- **Tập 2: Bài tập** gồm 42 bài, mỗi bài học có ba bài tập ở ba mức độ (cơ bản, trung bình, nâng cao), có ràng buộc theo subtask như đề thi thật.
- **Tập 3: Hướng dẫn giải** phân tích từng bài tập từ cách ngây thơ đến cách tối ưu, kèm chương trình hoàn chỉnh và bảng điểm của từng cách làm.

Mỗi bài học không đưa ra công thức ngay. Em sẽ được đặt trước một bài toán cụ thể, thử cách ngây thơ, đếm số phép tính để thấy chỗ bế tắc, rồi được dẫn từng bước để phá bế tắc. Mọi ví dụ đều được chạy tay trên hình vẽ; mọi chương trình trong sách đều đã được kiểm tra bằng cách đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu ngẫu nhiên.

Chương trình được viết bằng C++ theo chuẩn C++14, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến. Tên biến được viết đầy đủ để em đọc code như đọc một đoạn văn.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học đi theo cùng một con đường, giống như cách một người giỏi thật sự suy nghĩ khi gặp bài mới:

Phần	Nội dung
Khởi động	Một câu hỏi nhỏ, gần gũi để em bắt đầu nghĩ.
Bài toán	Đề bài cụ thể, có ví dụ và hình minh họa.
Ý tưởng ngây thơ	Cách làm đầu tiên ai cũng nghĩ ra, kèm sơ đồ khối và phần đếm số phép tính.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ chậm, và đặt câu hỏi dẫn tới ý tưởng mới.
Phá bế tắc	Từng bước: quan sát, trạng thái, công thức, chạy tay, chứng minh, chương trình.
Mở rộng	Các biến thể thường gặp trong đề thi.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Ghi nhớ	Tóm tắt những điều cốt lõi nhất.
Luyện tập	Liên hệ tới các bài tập ở Tập 2 và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
BÀI TOÁN	Đề bài chính thức.
CÔNG THỨC	Trạng thái và công thức quy hoạch động.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp, giải thích vì sao là $O(n)$, $O(n^2)$, ...
BẾ TẮC	Chỗ cách làm hiện tại không vượt qua được.
VÌ SAO ĐÚNG?	Lập luận chứng minh tính đúng đắn.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.

Lời khuyên khi học: Hãy đọc với giấy và bút bên cạnh. Mỗi khi gặp một bảng phương án, hãy che phần đáp án lại và tự điền trước. Mỗi khi gặp một công thức, hãy tự trả lời bốn câu hỏi: trạng thái là gì, cơ sở là gì, công thức chuyển là gì, đáp án ở đâu và điền theo thứ tự nào. Chỉ khi tự viết lại được chương trình mà không nhìn sách, em mới thật sự nắm được bài.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH HỌC VỚI TÀI LIỆU NÀY **3**

CHƯƠNG 1. NỀN TẢNG QUY HOẠCH ĐỘNG **5**

Bài 1. Từ đệ quy đến bảng phương án 6
Bài 2. Quy hoạch động trên dãy 22
Bài 3. Quy hoạch động trên lưới và truy vết 35

CHƯƠNG 2. CÁC MÔ HÌNH KINH ĐIỂN **47**

Bài 4. Cái túi 0/1 và cái túi không giới hạn 48
Bài 5. Cái túi mở rộng: Giới hạn số lượng, sức chứa khổng lồ 64
Bài 6. Dãy con tăng dài nhất 74
Bài 7. Quy hoạch động trên hai xâu 84

CHƯƠNG 3. QUY HOẠCH ĐỘNG NÂNG CAO **96**

Bài 8. Quy hoạch động trên đoạn 97
Bài 9. Quy hoạch động trạng thái tập con (bitmask) 109
Bài 10. Quy hoạch động chữ số 118
Bài 11. Quy hoạch động trên cây 126
Bài 12. Đếm, đồ thị có hướng, xác suất và kì vọng 136

CHƯƠNG 4. TỐI ƯU HÓA QUY HOẠCH ĐỘNG **146**

Bài 13. Tăng tốc bằng mảng cộng dồn và hàng đợi đơn điệu 147
Bài 14. Quy hoạch động với cây Henwick và lũy thừa ma trận 156

Lê Thanh Nam

CHƯƠNG 1. NỀN TẢNG QUY HOẠCH ĐỘNG

Người không rút ra bài học từ quá khứ sẽ dễ lặp lại những sai lầm đã qua.. Một chương trình cũng vậy: Nếu không ghi lại những gì đã tính, nó sẽ giải đi giải lại cùng một bài toán con hàng tỉ lần. Quy hoạch động là nghệ thuật chia bài toán lớn thành các bài toán con, giải mỗi bài toán con đúng một lần, ghi kết quả vào bảng và ghép chúng lại. Chương này xây nền móng cho cả cuốn sách: Em sẽ tự tay biến một chương trình đệ quy chạy hàng giờ thành một chương trình chạy trong chớp mắt, rồi học cách đặt bốn câu hỏi giúp em giải mọi bài quy hoạch động.

TRONG CHƯƠNG NÀY

Bài 1. Từ đệ quy đến bảng phương án: Quy hoạch động là gì?

Bài 2. Quy hoạch động trên dãy: "Kết thúc tại i" và "chọn hay bỏ"

Bài 3. Quy hoạch động trên lưới và truy vết đường đi

Lê Thành Nam

BÀI 1 Từ đệ quy đến bảng phương án

Học xong bài này, em sẽ:

✓ Thấy tận mắt vì sao đệ quy "ngây thơ" tính đi tính lại cùng một việc hàng tỉ lần.

✓ Nắm được con đường nâng cấp: Đệ quy → đệ quy có nhớ → điền bảng → tiết kiệm bộ nhớ.

✓ Trả lời được bốn câu hỏi của mọi bài quy hoạch động: Trạng thái, cơ sở, công thức chuyển, đáp án và thứ tự tính.

✓ Phân biệt bài toán đếm (cộng các cách) và bài toán tối ưu (lấy min hoặc max), biết vì sao tham lam có thể sai.

KHỞI ĐỘNG

Trước mặt em là một dãy đá bắc qua suối, đánh số từ 0 (bờ bên này) đến 5 (bờ bên kia). Mỗi lần em nhảy tới trước 1, 2 hoặc 3 hòn đá. Hòn đá có [illegible] rêu rất trơn, đặt chân lên là ngã. Có bao nhiêu cách qua suối?

Hãy lấy giấy bút liệt kê thử trước khi đọc tiếp. Rồi tưởng tượng dãy có một triệu hòn đá: Liệt kê là vô vọng, vì số cách lớn hơn số nguyên tử trong vũ trụ. Thế nhưng cuối bài này, em sẽ tính ra con số đó (lấy dư) trong chưa đầy một phần mười giây.

BÀI TOÁN. NHẢY ĐÁ [NHAYDA]

Có $n + 1$ hòn đá đánh số $0, 1, \ldots, n$. Em đứng ở đá 0 và muốn tới đá n. Mỗi bước em nhảy từ đá i tới đá $i + 1, i + 2$ hoặc $i + 3$. Có m hòn đá bị trơn, không được đặt chân lên. Hãy đếm số cách đi từ đá 0 tới đá n, in ra số dư khi chia cho $10^9 + 7$.

Dữ liệu vào:

- Dòng đầu gồm hai số n, m ($1 \le n \le 10^6, 0 \le m \le n$).

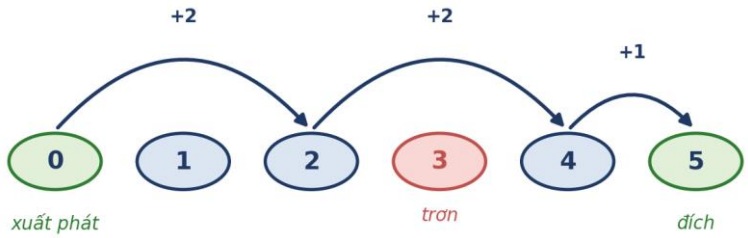
- Dòng thứ hai gồm m số nguyên phân biệt là vị trí các đá trơn (mỗi số thuộc đoạn $[1, n]$).

Kết quả:

- Số cách đi, lấy dư cho $10^9 + 7$.

Dữ liệu vào	Kết quả	Giải thích
5 1 3	5	Đá 3 trơn. Năm cách được liệt kê ở Hình 1.2.
4 0	7	Không có đá trơn: 7 cách lên đá 4.

Lê Thành Nam

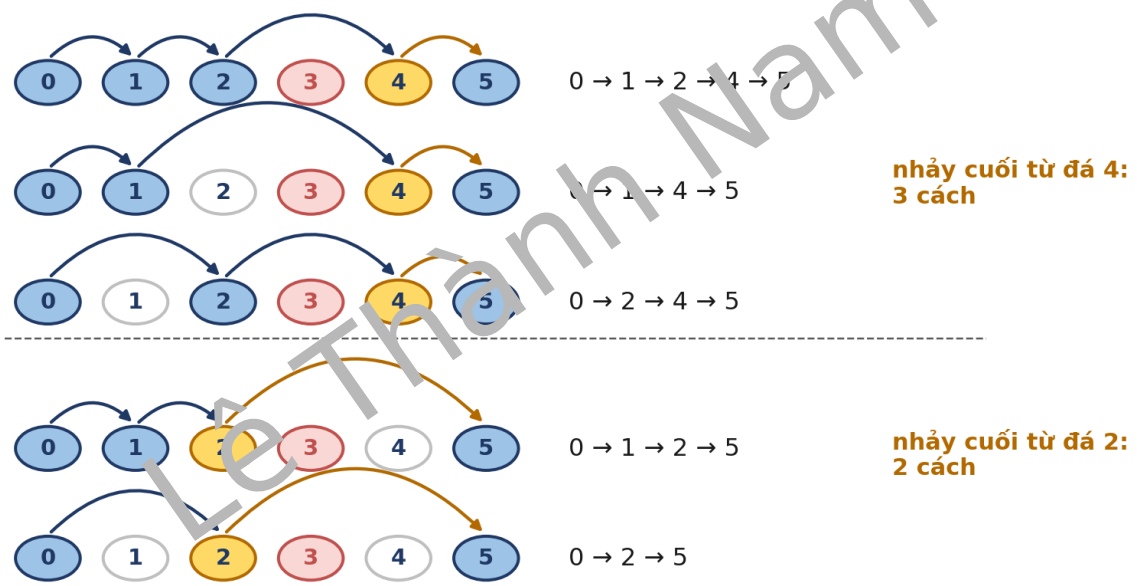


Hình 1.1. *Một cách qua suối: 0 → 2 → 4 → 5. Đá 3 (màu đỏ) bị trơn*

1. Ý tưởng ngây thơ: Liệt kê mọi cách

Cách tự nhiên nhất là thử mọi khả năng: Từ đá đang đứng, lần lượt thử nhảy 1, 2, 3 bước; gặp đá trơn hoặc vượt quá đá n thì bỏ; tới được đá n thì đếm thêm một cách.

Với ví dụ 1, ta liệt kê được đúng năm cách. Để không sót, ta xếp chúng theo hòn đá đứng ngay trước khi nhảy lên đá 5:



Hình 1.2. *Năm cách qua suối, xếp theo hòn đá cuối cùng trước khi tới đích (tô vàng)*

Lập bảng các trường hợp nhỏ

Trước khi nghĩ tới công thức, hãy làm như một nhà khoa học: Tính tay các trường hợp nhỏ, ghi vào bảng, rồi tìm quy luật.

Ví dụ thứ nhất (đá 3 trơn), ta liệt kê tất cả các cách đi tới từng hòn đá $i = 0, 1, \ldots, 5$ và đếm. Gọi số cách tới đá i là $f(i)$.

Lê Thành Nam

i	Liệt kê mọi cách tới đá i	f(i)	f(i − 1) + f(i − 2) + f(i − 3)
0	(đứng yên ở đá 0)	**1**	không xét
1	0→1	**1**	1 + 0 + 0 = 1
2	0→1→2; 0→2	**2**	1 + 1 + 0 = 2
3	không có: đá trơn	**0**	2 + 1 + 1 = 4 (nhưng đá trơn)
4	0→1→2→4; 0→1→4; 0→2→4	**3**	0 + 2 + 1 = 3
5	0→1→2→4→5; 0→1→2→5; 0→1→4→5; 0→2→4→5; 0→2→5	**5**	3 + 0 + 2 = 5

Hãy che cột cuối lại và chỉ nhìn cột $f(i)$: 1, 1, 2, 0, 3, 5. Các con số có vẻ lộn xộn. Bây giờ mở cột cuối ra: ở mọi hòn đá không trơn, tổng ba ô f liền trước đúng bằng f(i). Chẳng hạn ở đá 5: f(4) + f(3) + f(2) = 3 + 0 + 2 = 5, khớp với 5 cách đã liệt kê. Riêng đá 3 trơn thì f(3) = 0 dù tổng ba ô trước là 4.

Từ bảng, ta dự đoán công thức $f(i) = f(i-1) + f(i-2) + f(i-3)$ (và $f(i) = 0$ nếu đá i trơn). Một dự đoán chỉ đúng trên vài trường hợp thì chưa đủ; ta cần giải thích vì sao nó luôn đúng. Quan sát dưới đây làm việc đó.

Quan sát vàng: Nhìn vào bước nhảy cuối cùng

Mọi cách tới đá 5 đều phải đặt chân lần cuối lên đá 4, đá 3 hoặc đá 2 (vì mỗi bước nhảy tối đa 3 đá).

- Ba cách đầu kết thúc bằng bước 4 → 5. Bỏ bước cuối đi, ta được đúng tất cả các cách tới đá 4, không thừa không thiếu. Vậy có 3 cách.
- Không có cách nào đi qua đá 3, vì đá 3 trơn: 0 cách.
- Hai cách sau kết thúc bằng bước 2 → 5. Bỏ bước cuối, ta được tất cả các cách tới đá 2: 2 cách.

Vậy số cách tới đá 5 = (số cách tới đá 4) + (số cách tới đá 3) + (số cách tới đá 2) = 3 + 0 + 2 = 5. Bài toán lớn được ghép từ ba bài toán cùng dạng nhưng nhỏ hơn.

> **CÔNG THỨC TRUY HỒI**
>
> Gọi $f(i)$ là số cách đi từ đá 0 tới đá i. Xét bước nhảy cuối cùng: Trước khi tới đá i, em đứng ở đá $i-1, i-2$ hoặc $i-3$. Ba nhóm này không trùng nhau (khác nhau ở bước cuối) và gộp lại đủ mọi cách, nên:
>
> $$\boldsymbol{f(i) = f(i-1) + f(i-2) + f(i-3)}$$
>
> với quy ước: $\boldsymbol{f(0) = 1}$ (đứng yên ở đá 0 là một cách), f(i) = 0 nếu i < 0, và $\boldsymbol{f(i) = 0}$ **nếu đá i trơn**.

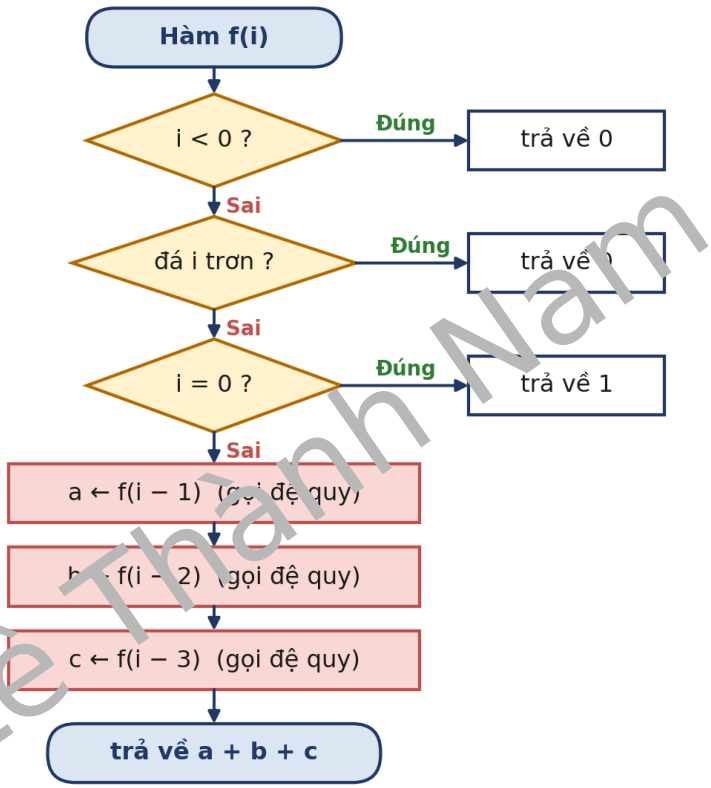
Cài đặt ngây thơ: Đệ quy trực tiếp

Công thức trên có thể chép gần như nguyên văn thành một hàm đệ quy:

Lê Thành Nam

Code ý chính: Đệ quy ngây thơ

```
long long go(int i) {
    if (i < 0) return 0;
    if (bad[i]) return 0;
    if (i == 0) return 1;
    long long a = go(i - 1);
    long long b = go(i - 2);
    long long c = go(i - 3);
    return a + b + c;
}
```



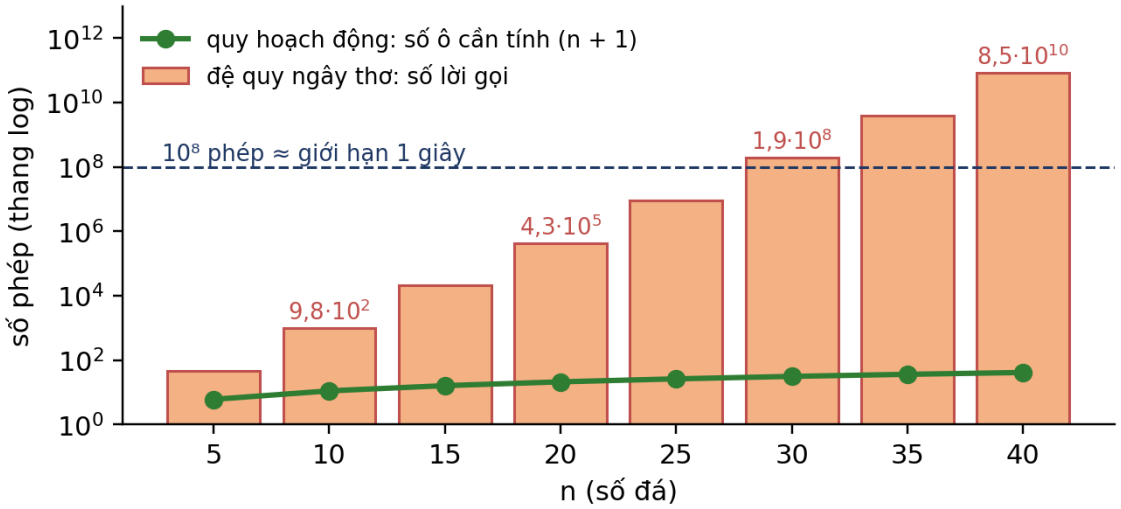
Lê Thành Nam

Hình 1.3. *Sơ đồ khối của hàm đệ quy ngây thơ f(i). Ba ô đỏ là ba lời gọi đệ quy*

Hàm chạy đúng: Với ví dụ 1 nó trả về 5, với ví dụ 2 trả về 7. Vậy vấn đề nằm ở đâu?

VÌ SAO CÁCH NGÂY THƠ CHẬM? ĐẾM TỪNG BƯỚC

- Gọi $C(i)$ là số lời gọi hàm khi tính $f(i)$ (giả sử không có đá trơn). Lời gọi $f(i)$ sinh thêm ba lời gọi $f(i-1), f(i-2), f(i-3)$, nên $C(i) = 1 + C(i-1) + C(i-2) + C(i-3)$.
- Dãy này tăng giống dãy Tribonacci: Mỗi khi i tăng 1, $C(i)$ tăng khoảng 1,84 lần. Vậy $C(n)$ cỡ $1{,}84^n$: Độ phức tạp hàm mũ $O(1{,}84^n)$.
- Đo thực tế: n = 20 cần khoảng $4{,}3 \cdot 10^5$ lời gọi; n = 30 cần $1{,}9 \cdot 10^8$ lời gọi (đã vượt 1 giây); n = 40 cần $8{,}5 \cdot 10^{10}$ lời gọi (khoảng mười lăm phút).



Hình 1.4. Số lời gọi của đệ quy ngây thơ (cột cam) bùng nổ theo hàm mũ, trong khi chỉ có n + 1 giá trị khác nhau cần tính (đường xanh)

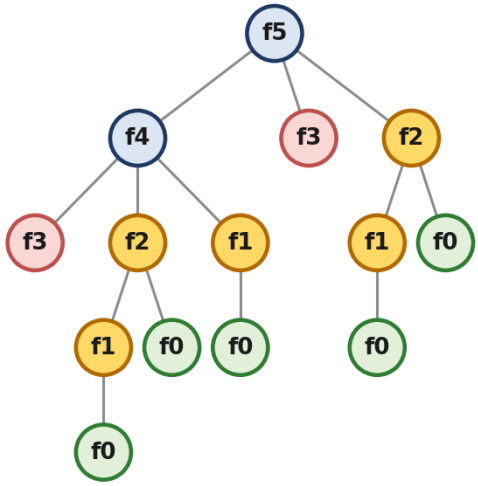
BẾ TẮC

Với n = 10^6, chương trình đệ quy sẽ không bao giờ chạy xong. Thế nhưng nhìn lại: Chỉ có n + 1 đại lượng khác nhau là f(0), f(1), ..., f(n). Vậy máy tính đã làm gì với hàng tỉ lời gọi còn lại?

2. Phá bế tắc

Bước 1 Vẽ cây lời gọi để tìm chỗ lãng phí

Mỗi lời gọi f(i) sinh ra các lời gọi con f(i − 1), f(i − 2), f(i − 3). Vẽ hết ra cho ví dụ n = 5, đá 3 trơn (bỏ qua các lời gọi với chỉ số âm):



Hình 1.5. Cây lời gọi của f(5). Nút vàng là bài toán con bị giải nhiều lần, nút đỏ là đá trơn

Lê Thành Nam

Chỉ với 5 hòn đá mà f(2) đã bị tính 2 lần, f(1) bị tính 3 lần, f(0) bị gọi 5 lần. Quan trọng hơn: Mỗi lần tính lại f(2) là một lần dựng lại nguyên cả cây con bên dưới nó. Với n lớn, các cây con trùng nhau chồng lên nhau theo cấp số nhân. Đây là dấu hiệu nhận biết số một của quy hoạch động:

> **DẤU HIỆU: BÀI TOÁN CON GỐI NHAU**
>
> Bài toán lớn được chia thành các bài toán con, và cùng một bài toán con xuất hiện ở nhiều nhánh khác nhau. Số bài toán con khác nhau thì ít (ở đây là n + 1), nhưng số lần gặp lại chúng thì khổng lồ.

Bước 2 Nhớ lại những gì đã tính (đệ quy có nhớ)

Ý tưởng rất đời thường: Tính xong f(i) thì **ghi vào sổ** `memo[i]`. Lần sau cần f(i), ta tra sổ thay vì tính lại. Ta dùng giá trị −1 để đánh dấu "chưa tính".

Code ý chính: Đệ quy có nhớ (memoization)

```
vector<long long> memo;
long long go(int i) {
    if (i < 0) return 0;
    if (bad[i]) return 0;
    if (i == 0) return 1;
    if (memo[i] != -1) return memo[i];
    long long a = go(i - 1);
    long long b = go(i - 2);
    long long c = go(i - 3);
    memo[i] = (a + b + c) % MOD;
    return memo[i];
}
```

Lê Thành Nam

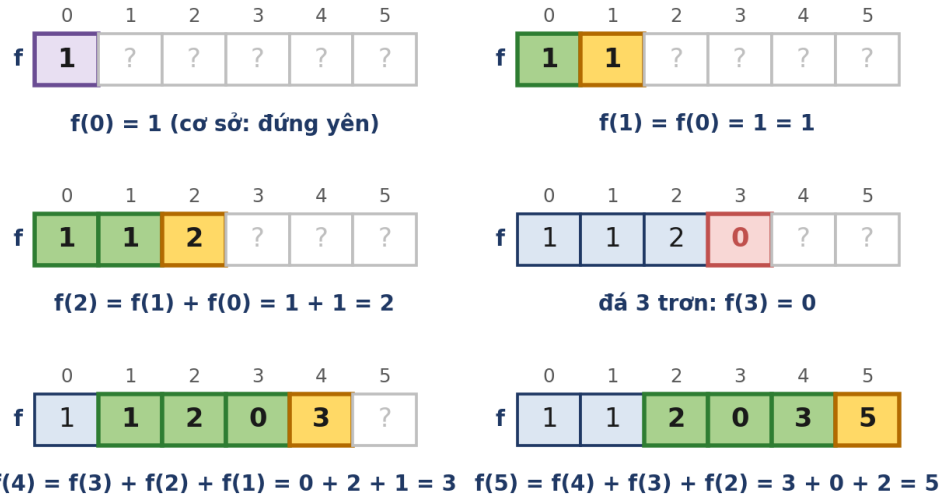
Mỗi giá trị f(i) giờ chỉ được **tính thật** đúng một lần; mọi lần gọi sau chỉ tốn một phép tra mảng. Có n + 1 giá trị, mỗi giá trị tính bằng 3 lời gọi con, nên tổng số lời gọi chỉ khoảng 3n: Độ phức tạp giảm từ $O(1{,}84^n)$ xuống **O(n)**. Với n = 40, số lời gọi giảm từ $8{,}5 \cdot 10^{10}$ xuống còn khoảng 120.

Tuy nhiên, khi gọi f(10^6), hàm sẽ gọi f(999 999), rồi f(999 998), ... chồng lên nhau **một triệu tầng** trước khi chạm tới f(0). Mỗi tầng chiếm một phần bộ nhớ ngăn xếp (stack), mà ngăn xếp mặc định trên nhiều máy chấm chỉ vài megabyte. Kết quả: Chương trình bị lỗi tràn ngăn xếp (stack overflow) ở test lớn.

Bước 3 Bỏ đệ quy: Điền bảng từ dưới lên

Hãy để ý: Để tính f(i) ta chỉ cần f(i − 1), f(i − 2), f(i − 3), đều là những ô có chỉ số nhỏ hơn. Vậy nếu ta tính lần lượt f(0), f(1), f(2), ... theo thứ tự tăng dần, thì khi tới ô i, ba ô cần dùng chắc chắn đã có sẵn. Không cần đệ quy nữa!

Hãy theo dõi kĩ từng bước điền bảng cho ví dụ thứ nhất (n = 5, đá 3 trơn). Quy ước màu trong các hình: ô vàng là ô đang điền; ô xanh lá là các ô mà nó dựa vào; ô xanh dương là các ô đã điền xong; ô trắng có dấu ? là các ô chưa điền. Ô màu tím là ô cơ sở, ô màu đỏ là đá trơn.



Hình 1.6. Điền bảng f từng ô theo thứ tự i = 0, 1, ..., 5

Toàn bộ quá trình được ghi lại trong bảng sau. Khi tự làm, em hãy kẻ đúng một bảng như thế này ra giấy: mỗi dòng là một ô, ghi rõ ô đó dựa vào những ô nào và phép tính cụ thể. Đây là cách chắc chắn nhất để không nhầm chỉ số.

Ô	Dựa vào các ô	Phép tính	Kết quả
f(0)	cơ sở	đứng yên là một cách	**1**
f(1)	f(0)	1	**1**
f(2)	f(1), f(0)	1 + 1	**2**
f(3)	đá trơn	không được đặt chân	**0**
f(4)	f(3), f(2), f(1)	0 + 2 + 1	**3**
f(5)	f(4), f(3), f(2)	3 + 0 + 2	**5**

Ở dòng f(1) và f(2), một vài số hạng như f(−1), f(−2) không có mặt vì chỉ số âm (không có hòn đá âm). Trong chương trình, điều này ứng với điều kiện `i - d >= 0`.

Code ý chính: Điền bảng từ dưới lên (bottom-up)

```
f[0] = 1;
for (int i = 1; i <= n; i++) {
    if (bad[i]) continue;
    for (int d = 1; d <= 3; d++)
        if (i - d >= 0)
            f[i] = (f[i] + f[i - d]) % MOD;
}
```

Bảng `f` được gọi là bảng phương án. Vòng lặp ngoài chạy n lần, vòng trong 3 lần, nên độ phức tạp là O(3n) = O(n): Với $n = 10^6$ chỉ khoảng $3 \cdot 10^6$ phép cộng, chạy trong vài mili giây, và không còn tầng đệ quy nào.

Lê Thành Nam

Bước 4 Tiết kiệm bộ nhớ: Chỉ nhớ ba ô gần nhất

Khi tính tới ô i, các ô từ i − 4 trở về trước không bao giờ được dùng lại. Vậy ta chỉ cần giữ ba giá trị gần nhất và "trượt" chúng sang phải sau mỗi bước:



Hình 1.7. Cửa sổ trượt ba ô: Bộ nhớ cho bảng phương án giảm từ O(n) xuống O(1)

Code ý chính: Ba biến trượt

```
long long a = 0, b = 0, c = 1;
for (int i = 1; i <= n; i++) {
    long long cur = (c + b + a) % MOD;
    if (bad[i]) cur = 0;
    a = b;
    b = c;
    c = cur;
}
```

Ở bài này, mảng đánh dấu đá trơn vẫn tốn O(n) bộ nhớ. Cải tiến này chưa thật cần. Nhưng ở những bài bảng phương án hai chiều (Bài 3, Bài 4, Bài 7), kĩ thuật "chỉ giữ vài hàng gần nhất" có thể là ranh giới giữa được điểm và bị tràn bộ nhớ.

Cách làm	Thời gian	Bộ nhớ	Ghi chú
Đệ quy ngây thơ	$O(1,84^n)$	O(n) ngăn xếp	Chỉ chạy được $n \le 25$
Đệ quy có nhớ	O(n)	O(n) + O(n) ngăn xếp	Dễ viết, có thể tràn ngăn xếp
Điền bảng	O(n)	O(n)	Nên dùng
Ba biến trượt	O(n)	O(1) cho bảng	Khi bộ nhớ eo hẹp

Lê Thành Nam

3. Quy hoạch động là gì?

Điều em vừa làm có tên là quy hoạch động (QHĐ, tiếng Anh: dynamic programming): Chia bài toán thành các bài toán con cùng dạng nhưng nhỏ hơn, giải mỗi bài toán con đúng một lần, lưu kết quả vào bảng và dùng lại khi cần. Quy hoạch động phát huy sức mạnh khi các bài toán con gối nhau (trùng lặp rất nhiều) như cây lời gọi ở Hình 1.5.

Mọi lời giải quy hoạch động, dù đơn giản hay phức tạp, đều được xây từ bốn câu trả lời. Hãy tập thói quen viết bốn câu trả lời này ra giấy trước khi viết dòng code đầu tiên.

BỐN CÂU HỎI CỦA MỌI BÀI QUY HOẠCH ĐỘNG

1. Trạng thái: f(...) nghĩa là gì? Phải phát biểu thành một câu rõ ràng, đầy đủ. Ở đây: "f(i) là số cách đi từ đá 0 tới đá i".

2. Cơ sở: Những trạng thái nhỏ nhất nào biết ngay đáp án mà không cần tính? Ở đây: f(0) = 1, f(âm) = 0, f(đá trơn) = 0.

3. Công thức chuyển: Tính một trạng thái từ những trạng thái nhỏ hơn như thế nào? Câu hỏi gợi ý tốt nhất là **"bước cuối cùng là gì?"**. Ở đây: f(i) = f(i − 1) + f(i − 2) + f(i − 3).

4. Đáp án và thứ tự tính: Đáp án nằm ở ô nào (ở đây f(n))? Điền bảng theo thứ tự nào để ô cần dùng luôn có sẵn (ở đây i tăng dần)?

VÌ SAO CÔNG THỨC ĐẾM ĐÚNG?

Một công thức đếm bằng cách cộng các nhóm chỉ đúng khi hai điều kiện cùng thỏa mãn:

- **Không trùng:** Mỗi cách đi chỉ rơi vào đúng một nhóm. Đúng vậy, vì mỗi cách đi có đúng một bước nhảy cuối, nên đúng một hòn đá đứng trước đá i.
- **Không sót:** Mọi cách đi đều rơi vào một nhóm nào đó. Đúng vậy, vì bước nhảy cuối chỉ có thể dài 1, 2 hoặc 3.

Ngoài ra, mỗi cách tới đá i − d ghép thêm bước nhảy d luôn cho một cách hợp lệ tới đá i (nếu đá i không trơn), và ngược lại. Đó là một **song ánh** giữa nhóm thứ d và tập các cách tới đá i − d, nên nhóm thứ d có đúng f(i − d) phần tử.

Lê Thanh Nam

4. Từ đếm sang tối ưu: Bài toán PHIQUA

Quy hoạch động không chỉ dùng để đếm. Hãy giữ nguyên dãy đá nhưng đổi câu hỏi.

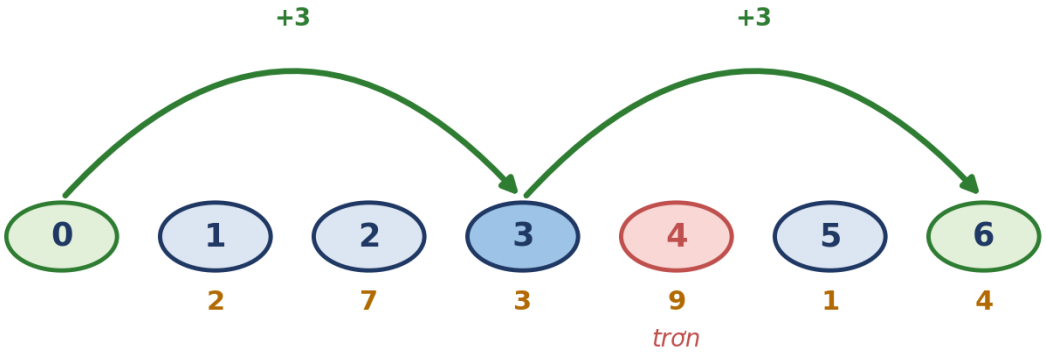
BÀI TOÁN PHÍ QUA: [PHIQUA]

Vẫn n + 1 hòn đá đánh số 0, 1, ..., n và mỗi bước nhảy 1, 2 hoặc 3 đá. Mỗi khi đặt chân lên đá i ($1 \le i \le n$) em phải trả phí c_i; $c_i = -1$ nghĩa là đá i trơn. Tìm tổng phí nhỏ nhất để đi từ đá 0 tới đá n, hoặc in −1 nếu không thể tới.

Dữ liệu vào: Dòng đầu là n ($1 \le n \le 10^6$). Dòng thứ hai gồm n số $c_1, c_2, ..., c_n$ ($c_i = -1$ hoặc $0 \le c_i \le 10^9$).

Kết quả: Tổng phí nhỏ nhất, hoặc −1.

Dữ liệu vào	Kết quả	Giải thích
6 2 7 3 -1 1 4	7	Đi 0 → 3 → 6, trả 3 + 4 = 7.

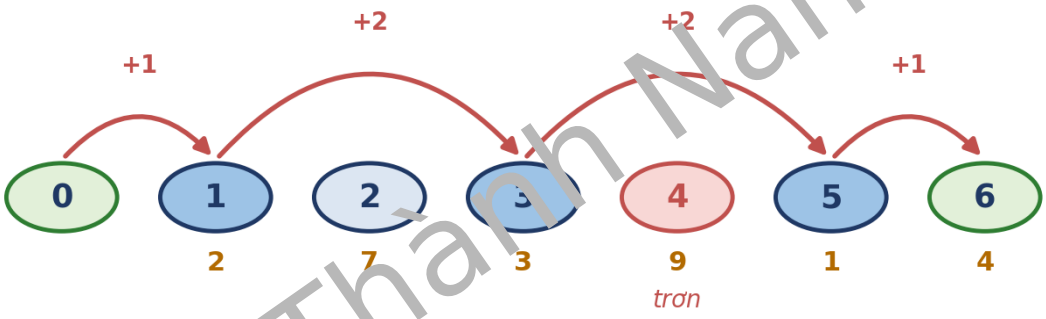


phí ghi màu cam dưới mỗi đá (trả khi đặt chân); đường xanh: 0 → 3 → 6, tổng phí 3 + 4 = 7

Hình 1.8. *Ví dụ của bài PHIQUA: Đường đi tốt nhất có tổng phí 7*

Một cách sai hấp dẫn: Tham lam

Ý tưởng đầu tiên của nhiều bạn: "Mỗi bước, nhảy tới hòn đá có phí rẻ nhất trong ba hòn phía trước". Hãy thử với ví dụ:



tham lam "đá kế tiếp rẻ nhất": 0 → 1 → 3 → 5 → 6, tổng phí 2 + 3 + 1 + 4 = 10 (sai)

Hình 1.9. *Tham lam chọn đá rẻ nhất trước mắt: Tổng phí 10, kém hơn đáp án đúng 7*

Ở bước đầu tiên, tham lam chọn đá 1 (phí 2) vì rẻ hơn đá 3 (phí 3). Nhưng chính lựa chọn "rẻ trước mắt" đó buộc em phải đi thêm nhiều bước sau. Bài học: **Một quyết định tốt ở hiện tại chưa chắc dẫn tới kết quả tốt nhất ở cuối.** Quy hoạch động không đoán; nó xét **mọi** khả năng một cách có hệ thống nhưng không lặp lại công việc.

Lập bảng các trường hợp nhỏ

Lại bắt đầu bằng việc tính tay. Với mỗi hòn đá i, liệt kê mọi đường đi tới i cùng tổng phí (số trong ngoặc), rồi chọn đường rẻ nhất. Gọi phí nhỏ nhất tới đá i là g(i).

Lê Thành Nam

i	Các đường tới đá i (tổng phí)	g(i)	min(ba ô trước) + c_i
1	0→1 (2)	**2**	min(0) + 2 = 2
2	0→1→2 (9); 0→2 (7)	**7**	min(2, 0) + 7 = 7
3	0→1→2→3 (12); 0→1→3 (5); 0→2→3 (10); 0→3 (3)	**3**	min(7, 2, 0) + 3 = 3
4	không có: đá trơn	∞	đá trơn
5	0→1→2→3→5 (13); 0→1→2→5 (10); 0→1→3→5 (6); 0→2→3→5 (11); 0→2→5 (8); 0→3→5 (4)	**4**	min(∞, 3, 7) + 1 = 4
6	0→1→2→3→5→6 (17); 0→1→2→3→6 (16); 0→1→2→5→6 (14); 0→1→3→5→6 (10); 0→1→3→6 (9); 0→2→3→5→6 (15); 0→2→3→6 (14); 0→2→5→6 (12); 0→3→5→6 (8); 0→3→6 (7)	**7**	min(4, ∞, 3) + 4 = 7

Nhìn cột g(i) và cột cuối: ở mọi đá không trơn, g(i) đúng bằng phí nhỏ nhất trong ba ô liền trước, cộng thêm phí của chính đá i.

Chẳng hạn g(5) = min(g(4), g(3), g(2)) + 1 = min(∞, 3, 7) + 1 = 4: trong 6 đường tới đá 5, đường rẻ nhất 0→3→5 đi qua đá 3, và đoạn 0→3 của nó cũng chính là đường rẻ nhất tới đá 3. Bảng gợi ý công thức; các câu hỏi dưới đây phát biểu và giải thích nó.

Bốn câu hỏi cho PHIQUA

1. Trạng thái: g(i) là tổng phí nhỏ nhất để đi từ đá 0 tới đá i (bằng ∞ nếu không tới được).

2. Cơ sở: g(0) = 0, g(i) = ∞ nếu đá i trơn.

3. Công thức chuyển: Bước cuối cùng tới đá i xuất phát từ đá i − 1, i − 2 hoặc i − 3. Đã tới đá i − d với phí nhỏ nhất g(i − d) thì cộng thêm phí c_i. Lấy nhỏ nhất trong ba khả năng:

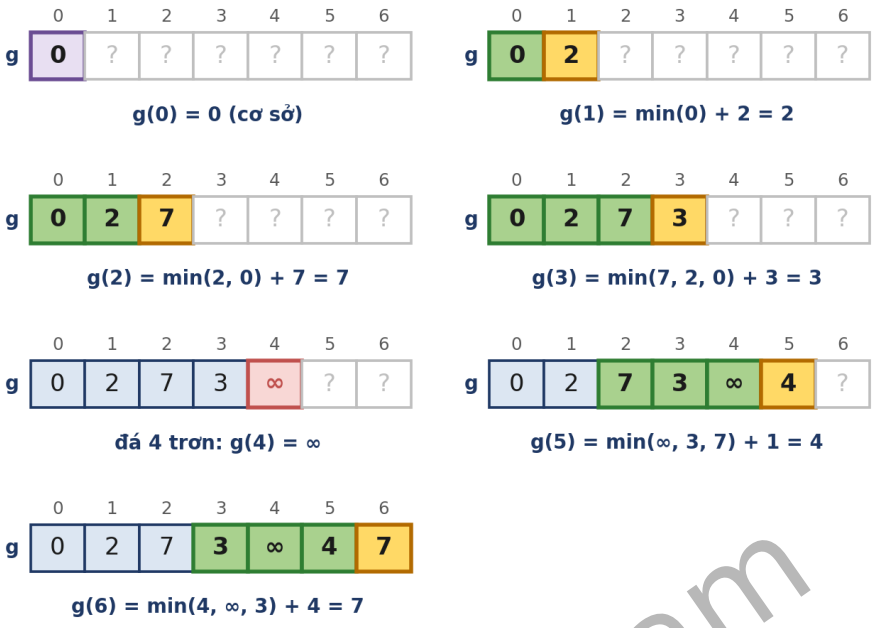
$$\mathbf{g(i) = \min(g(i-1), g(i-2), g(i-3)) + c_i}$$

4. Đáp án và thứ tự tính: g(n), hoặc −1 nếu g(n) = ∞; điền i tăng dần.

So sánh với bài đếm: Cấu trúc giống hệt, chỉ khác phép toán. Bài đếm dùng phép cộng để gộp các nhóm; bài tối ưu dùng phép min để chọn nhóm tốt nhất.

Điền bảng g từng ô cho ví dụ (đá 4 trơn). Quy ước màu trong các hình: ô vàng là ô đang điền; ô xanh lá là các ô mà nó dựa vào; ô xanh dương là các ô đã điền xong; ô trắng có dấu ? là các ô chưa điền.

Lê Thành Nam



Hình 1.10. Điền bảng g từng ô. Ô đỏ là đá trơn, giá trị ∞

Ô	Dựa vào	Phép tính	Kết quả
g(0)	cơ sở	cứa trả phí	**0**
g(1)	g(0)	min(0) + 2 = 0 + 2	**2** (từ đá 0)
g(2)	g(1), g(0)	min(2, 0) + 7 = 0 + 7	**7** (từ đá 0)
g(3)	g(2), g(1), g(0)	min(7, 2, 0) + 3 = 0 + 3	**3** (từ đá 0)
g(4)	đá trơn	không tới được	∞
g(5)	g(4), g(3), g(2)	min(∞, 3, 7) + 1 = 3 + 1	**4** (từ đá 3)
g(6)	g(5), g(4), g(3)	min(4, ∞, 3) + 4 = 3 + 4	**7** (từ đá 3)

Cột "Kết quả" ghi thêm hòn đá đạt giá trị min. Những mũi tên "từ đá ..." này chính là thông tin để truy vết đường đi tốt nhất: từ đá 6 lùi về đá 3, từ đá 3 lùi về đá 0. Bài 3 sẽ học kĩ thuật truy vết.

> **VÌ SAO LẤY MIN LÀ ĐÚNG? CẤU TRÚC CON TỐI ƯU**
>
> Xét một đường đi tốt nhất tới đá i, và giả sử bước cuối nhảy từ đá j. Khi đó đoạn đường từ 0 tới j cũng phải là đường đi tốt nhất tới j. Thật vậy, nếu có đường khác tới j rẻ hơn, ta thay đoạn đầu bằng đường rẻ hơn đó rồi nhảy j → i, sẽ được đường tới i rẻ hơn, mâu thuẫn với giả thiết "tốt nhất".
>
> Tính chất "lời giải tốt nhất của bài lớn chứa lời giải tốt nhất của bài con" gọi là cấu trúc con tối ưu. Bài toán có cấu trúc con tối ưu và bài toán con gối nhau là hai dấu hiệu để dùng quy hoạch động cho bài tối ưu.

Lê Thành Nam

5. Chương trình hoàn chỉnh

Dưới đây là chương trình hoàn chỉnh cho hai bài. Phần đọc tệp được viết theo kiểu "có tệp thì đọc tệp": Khi chấm bằng Themis với tệp NHAYDA.INP, chương trình đọc và ghi tệp; khi nộp lên trang chấm trực tuyến (đọc từ bàn phím), điều kiện `fopen` sai nên chương trình đọc chuẩn như bình thường.

NHAYDA.CPP: Điền bảng, O(n) thời gian

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

const long long MOD = 1000000007LL;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("NHAYDA.INP", "r")) {
        freopen("NHAYDA.INP", "r", stdin);
        freopen("NHAYDA.OUT", "w", stdout);
    }
    int n, m;
    cin >> n >> m;
    vector<bool> bad(n + 1, false);
    for (int id = 0; id < m; id++) {
        int p;
        cin >> p;
        bad[p] = true;
    }
    vector<long long> f(n + 1, 0);
    f[0] = 1;
    for (int i = 1; i <= n; i++) {
        if (bad[i]) continue;
        for (int d = 1; d <= 3; d++) {
            if (i - d < 0) continue;
            f[i] = (f[i] + f[i - d]) % MOD;
        }
    }
    cout << f[n] << '\n';
    return 0;
}
```

Lê Thành Nam

Vài chi tiết đáng chú ý:

- Mảng `f` có n + 1 phần tử để dùng được chỉ số từ 0 đến n đúng như trong công thức.
- Phép lấy dư được thực hiện **ngay sau mỗi phép cộng**, nên mọi giá trị luôn nhỏ hơn $2 \cdot 10^9 + 7$, vừa kiểu long long thoải mái. Nếu đợi cộng xong mới lấy dư, các số sẽ tràn rất nhanh (f(100) đã có khoảng 26 chữ số).
- Đá trơn được bỏ qua bằng `continue`, nên `f[i]` giữ nguyên giá trị 0.

PHIQUA.CPP: Quy hoạch động tối ưu, O(n) thời gian

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

const long long INF = 1000000000000000000LL;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("PHIQUA.INP", "r")) {
        freopen("PHIQUA.INP", "r", stdin);
        freopen("PHIQUA.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    vector<long long> c(n + 1, 0);
    for (int i = 1; i <= n; i++) cin >> c[i];
    vector<long long> f(n + 1, INF);
    f[0] = 0;
    for (int i = 1; i <= n; i++) {
        if (c[i] < 0) continue;
        for (int d = 1; d <= 3; d++) {
            if (i - d < 0 || f[i - d] >= INF) continue;
            long long x = f[i - d] + c[i];
            f[i] = min(f[i], x);
        }
    }
    if (f[n] >= INF) cout << -1 << '\n';
    else cout << f[n] << '\n';
    return 0;
}
```

Ở bài tối ưu, ta dùng hằng số `INF` = 10^{18} để biểu diễn ∞. Câu lệnh kiểm tra `f[i - d] < INF` đảm bảo không bao giờ cộng phí vào một ô "không tới được"; nhờ vậy ô ∞ luôn giữ đúng giá trị ∞ và không bị tràn số.

Lê Thành Nam

6. Mở rộng

Mở rộng 1: Tập bước nhảy tùy ý

Nếu mỗi bước được nhảy một độ dài thuộc tập S = {s_1, s_2, ..., s_k}, công thức trở thành f(i) = Σ f(i − s) với s ∈ S. Độ phức tạp O(n·k). Khi các bước nhảy là mọi độ dài từ 1 đến k, tổng này là tổng của k ô liên tiếp, có thể tính trong O(1) nhờ mảng cộng dồn, đưa độ phức tạp về O(n) (xem Bài 13).

Mở rộng 2: Thứ tự có quan trọng không?

Ở bài NHAYDA, hai cách 0 → 1 → 3 và 0 → 2 → 3 được tính là khác nhau vì thứ tự bước nhảy khác nhau. Nếu đề bài hỏi "có bao nhiêu cách viết n thành tổng các số 1, 2, 3 không kể thứ tự", ta cần một trạng thái khác, đó là bài toán cái túi đếm tổ hợp ở Bài 4. Đọc kĩ đề để biết thứ tự có được tính hay không là một kĩ năng sống còn.

Mở rộng 3: n khổng lồ

Nếu n lên tới 10^{18} (không có đá trơn), ta không thể điền 10^{18} ô. Công thức f(i) = f(i − 1) + f(i − 2) + f(i − 3) là một truy hồi tuyến tính, có thể tính bằng lũy thừa ma trận 3×3 trong O(27·log n) phép nhân. Kĩ thuật này được trình bày ở Bài 14.

Mở rộng 4: Khi nào nên dùng đệ quy có nhớ?

Điền bảng nhanh hơn và an toàn hơn, nhưng đòi hỏi em biết trước thứ tự tính. Đệ quy có nhớ tự động "đi đúng thứ tự" và chỉ tính những trạng thái thật sự cần. Nó hữu ích khi thứ tự tính khó xác định (ví dụ quy hoạch động trên đồ thị ở Bài 12, quy hoạch động chữ số ở Bài 10) hoặc khi chỉ một phần nhỏ trạng thái được dùng tới. Hãy nhớ rằng độ sâu đệ quy khoảng 10^5 tầng trở lên là đã có nguy cơ tràn ngăn xếp.

BẪY THƯỜNG GẶP

- Chỉ số âm: Truy cập `f[i - 3]` khi i < 3 là lỗi truy cập ngoài mảng. Luôn kiểm tra `i - d >= 0`.
- Quên lấy dư hoặc lấy dư muộn: Kết quả bị tràn số, sai ngay từ n khoảng 70.
- Quên cơ sở f(0) = 1: Nhiều bạn đặt f(0) = 0 với lí do "chưa nhảy bước nào", và toàn bộ bảng bằng 0.
- Đá n trơn hoặc đá 0: Nếu đá n trơn thì đáp án là 0; chương trình trên xử lí đúng vì `f[n]` không được cộng.
- Cộng ∞ với phí: Ở bài tối ưu, cộng phí vào ô ∞ có thể tràn số hoặc tạo ra giá trị "giả" nhỏ hơn ∞. Luôn kiểm tra ô nguồn trước khi cộng.

GHI NHỚ

- Quy hoạch động = chia thành bài toán con + giải mỗi bài một lần + lưu bảng.
- Dấu hiệu: Bài toán con gối nhau; với bài tối ưu, thêm cấu trúc con tối ưu.
- Con đường nâng cấp: Đệ quy ngây thơ (hàm mũ) → đệ quy có nhớ (O(số trạng thái × số chuyển)) → điền bảng (không đệ quy) → giữ vài ô gần nhất (tiết kiệm bộ nhớ).
- Câu hỏi then chốt để tìm công thức: "Bước cuối cùng là gì?"

LUYỆN TẬP

Bài	Tên	Gợi ý liên hệ
1.1	LÁT SÀN	Đếm cách lát nền 2 × n bằng gạch 1 × 2 và 2 × 2. Xét viên gạch nằm ở cuối hàng.
1.2	MÃ HÓA	Đếm cách giải mã một xâu chữ số thành chữ cái. Bước cuối là một hay hai chữ số? Coi chừng chữ số 0.
1.3	MUA VÉ	Mỗi người mua vé một mình hoặc mua chung với người đứng sau. Bài tối ưu: Lấy min.

Lê Thành Nam

Tự kiểm tra nhanh

1. Tính f(6) của bài NHAYDA khi đá 2 và đá 4 trơn. *(Đáp số: 4. Bảng f = 1, 1, 0, 2, 0, 2, 4.)*

2. Không có đá trơn, mỗi bước nhảy 1 hoặc 2. Tính số cách tới đá 10. *(Đáp số: 89)*

3. Với ví dụ PHIQUA, nếu đá 3 cũng trơn thì đáp án là bao nhiêu? *(Đáp số: 12, đi 0 → 2 → 5 → 6. Bảng g = 0, 2, 7, ∞, ∞, 8, 12.)*

Lê Thành Nam